Grafana for Developers

A hands-on guide to observability, dashboards, and plugin development

2-Day Training · GitHub Codespaces · Prometheus · InfluxDB · Grafana OSS

Grafana

Hello

ธีรเศรษฐ์ จิรภัทร์ชาญเดช (อ.พล)

Microsoft® MVP Most Valuable Professional

- Founder, Nextflow.in.th
- Community Leader, Global AI Bangkok
- AI Instructor & Consultant

  – SCBx, Boon Rawd Brewery, สำนักงานนวัตกรรมแห่งชาติ, U.S. Embassy Bangkok

Contact me 😎

https://bit.ly/nf-grafana-dev

Course Overview

9 sections · 9 hands-on labs · one shared Codespaces environment

1 **Docker Basics**
Containers, Compose, essential commands

2 **Introduction to Grafana**
Architecture, capabilities, first install

3 **Working with Data Sources**
Prometheus, InfluxDB, connection patterns

4 **PromQL Basics**
Selectors, functions, aggregations

5 **Queries and Panels**
Explore → Panel → Dashboard

6 **Advanced Dashboard Features**
Variables, templating, drill-down

7 **Working with Alerting**
Rules, thresholds, state transitions

8 **Grafana Plugins**
Plugin types, scaffold, develop, live reload

9 **Summary & Next Steps**
Recap and open Q&A

Section 1

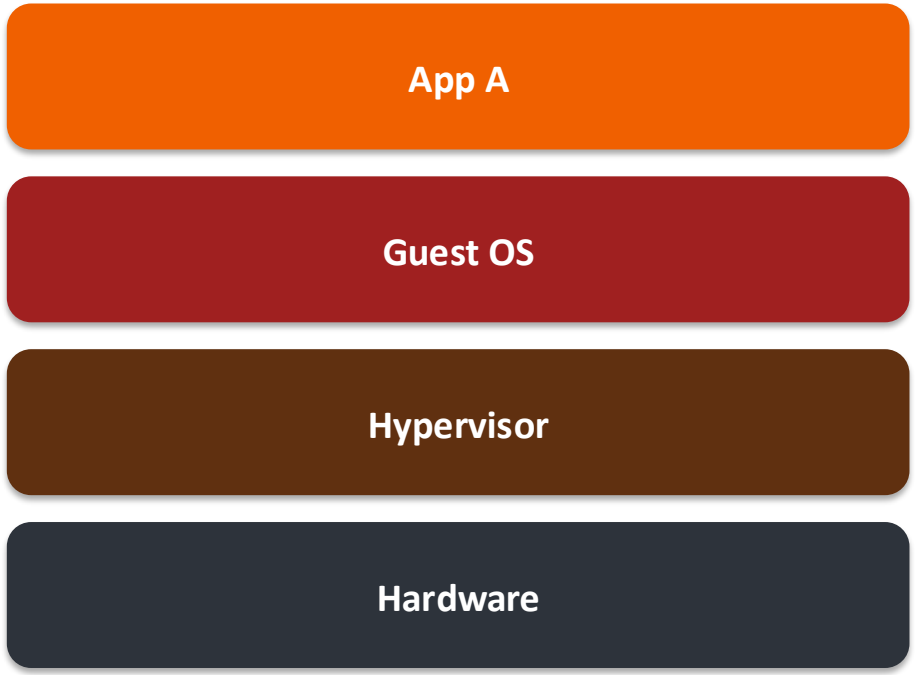
Docker Basics

Containers · Images · Compose

How to run an app?

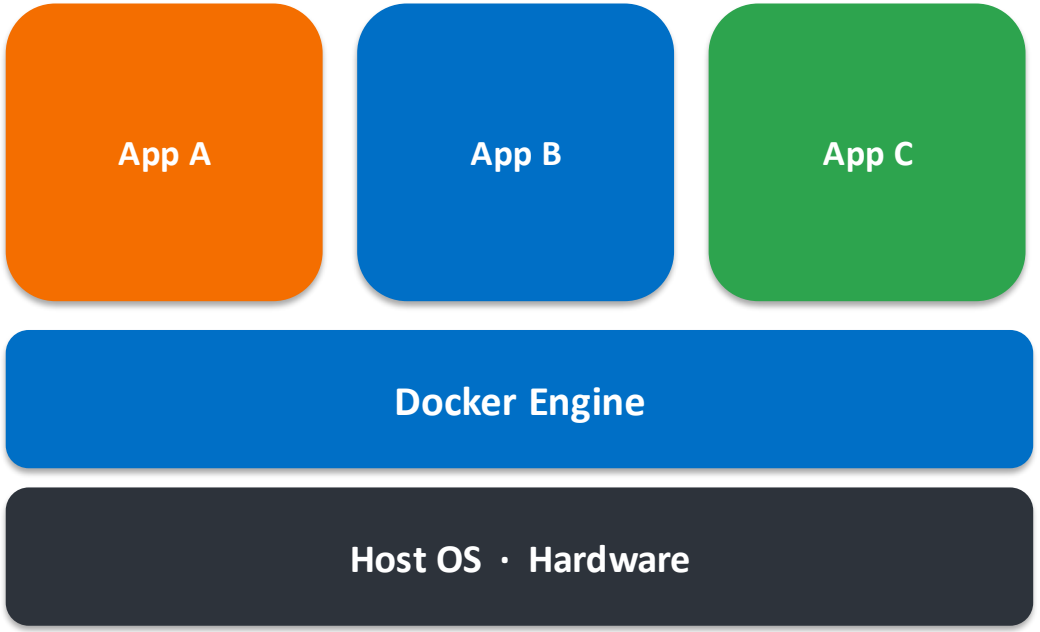
What is Docker and Containers?

Virtual Machine



vs

Containers



Containers share the host OS kernel — no Guest OS overhead, faster startup, lower resource use

Container

How Containers Work

Processes, not virtual machines

- Containers share the host OS kernel — no Guest OS overhead
- Each container has its own isolated process, network, and filesystem
- Many containers run simultaneously on a single machine
- Much faster to start and stop than full virtual machines

Key Components

Image · Container · Volume · Network

Blueprint / template
Read-only layers

Image

Live running instance
Created from image

Container

Persistent data
Survives restarts

Volume

Service-to-service
communication

Network

These four concepts appear in every Docker Compose file throughout the course.

Docker Desktop

Local GUI — same engine, different surface

- GUI for managing containers, images, and volumes on your laptop
- Uses the same Docker engine as GitHub Codespaces
- This course runs in Codespaces so everyone starts from the same baseline
- Docker Desktop is what learners will use after the course for local work

docker desktop PERSONAL

Search ⌘K

Sign in

- Gordon BETA
- Containers
- Images
- Volumes
- Kubernetes
- Builds
- Models
- MCP Toolkit BETA
- Docker Hub
- Docker Scout

- Extensions

Containers

Give feedback

Container CPU usage

0.00% / 1400% (14 CPUs available)

Container memory usage

0B / 7.57GB

Show charts

Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Last	Actions
	03-influxdb-flux	-	-	-	0%	1 da	

Walkthroughs

Multi-container applications

8 mins

Containerize your application

$ docker init

3 mins

View more in the Learning center

Engine running

RAM 1.92 GB CPU 0.14% Disk: 1.00 GB used (limit 910.74 GB)

v4.71.0

Essential Docker Commands

Your debugging survival kit

- docker ps — list running containers
- docker images — list locally available images
- docker logs <name> — view container log output
- docker exec -it <name> sh — open an interactive shell
- docker inspect <name> — view container metadata

Docker Compose

Docker Compose for Multi-Service Stacks

One file · all services · one command

docker-compose.yml

Grafana :3000

Prometheus :9090

InfluxDB :8086

Docker Internal Network (compose_default)

$ docker compose up -d

$ docker compose down

This same pattern is used in every lab exercise throughout the course.

Section 2

Introduction to Grafana

Explore · Dashboards · Alerting · Plugins

What is Grafana?

Visualize. Explore. Alert.

- Open-source observability and data visualization platform
- Connects to many data source types: Prometheus, InfluxDB, Loki, and more
- Central place for querying, visualizing, and answering questions from data
- Used by developers, SREs, and platform teams worldwide

Core Capabilities of Grafana

The four pillars of this course

Experiment with queries, view live data

🔍 **Explore**

Organize data for fast decision-making

📊 **Dashboards**

Detect important conditions automatically

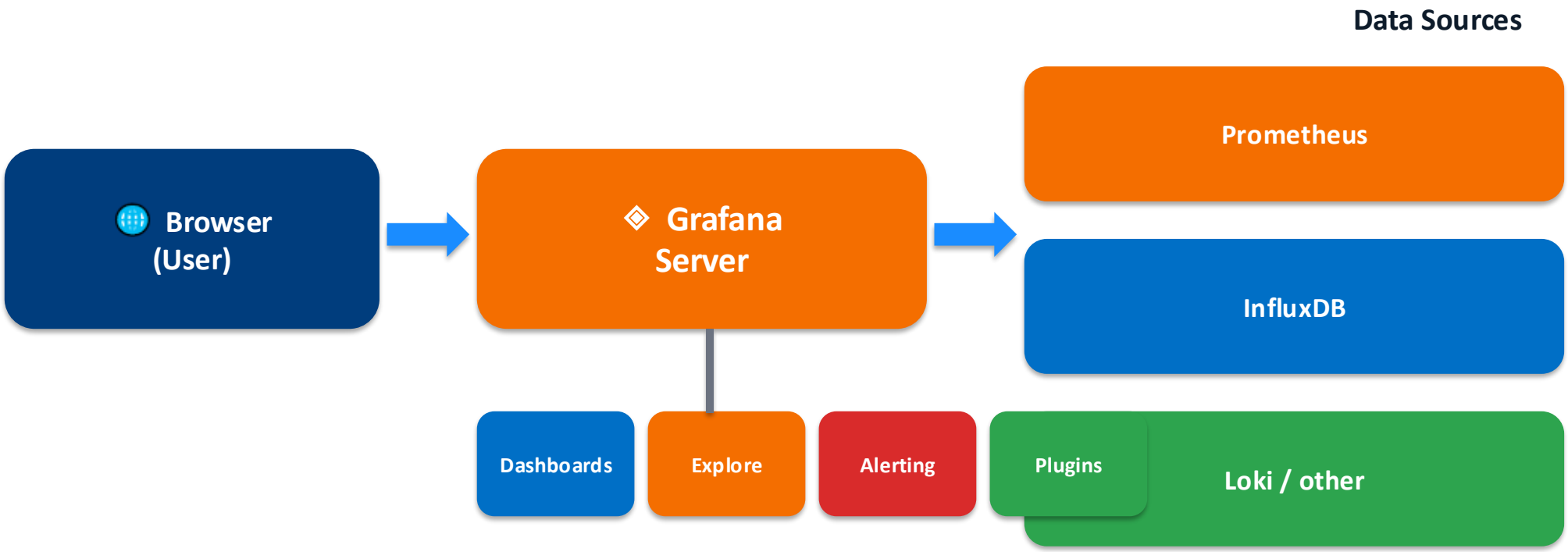
🔔 **Alerting**

Extend Grafana with custom capabilities

🔌 **Plugins**

Basic Grafana Architecture

Browser → Grafana → Data Sources



Grafana never stores metrics itself — it queries external systems and renders the results.

Section 3

Installation & Configuration

GitHub Codespaces · docker compose up · Port 3000

Preparing the Environment

GitHub Codespaces as the shared lab platform

- GitHub Codespaces provides an identical environment for every learner
- Docker and Docker Compose are pre-installed — no local setup required
- Port forwarding automatically exposes services like Grafana on port 3000
- Key files in every lab: `.env` and `docker-compose.yml`

Installing and Running Grafana

Start → Check → Open → Log In

- cd labfiles/01-setup-grafana-codespaces/
- docker compose up -d (start all services in the background)
- docker compose ps (verify all containers are Running)
- Open port 3000 in the Codespaces Ports panel
- Log in with: admin / admin and change password on first login

Setup Grafana in Codespaces

STEPS

1. Clone the repo and open it in GitHub Codespaces
2. Run docker compose up -d in labfiles/01-setup-grafana-codespaces/
3. Forward port 3000 and open Grafana in the browser
4. Inspect the Docker volume to understand data persistence

EXPECTED OUTCOMES

- ✓ Grafana running and accessible on port 3000
- ✓ Login with admin credentials confirmed
- ✓ Port forwarding understood
- ✓ Persistent Docker volume observed

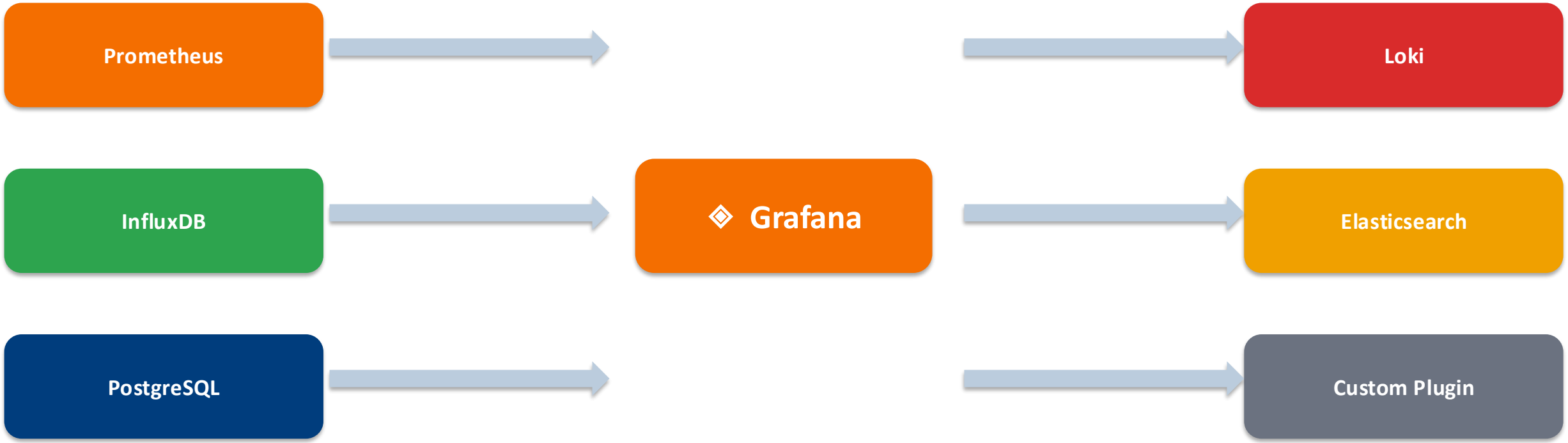
Section 4

Working with Data Sources

Prometheus · InfluxDB · Connection Patterns

What is a Data Source?

The connection point between Grafana and your data



All data sources follow the same Grafana pattern: pick target → enter credentials → test connection → query.

Prometheus Data Source

Time-series metrics · PromQL

- Prometheus scrapes and stores time-series metrics from configured targets
- Query language: PromQL — functional, built for time-series aggregation
- In Docker Compose, the URL is: http://prometheus:9090
- Explore → Code mode → type PromQL to see live metric data

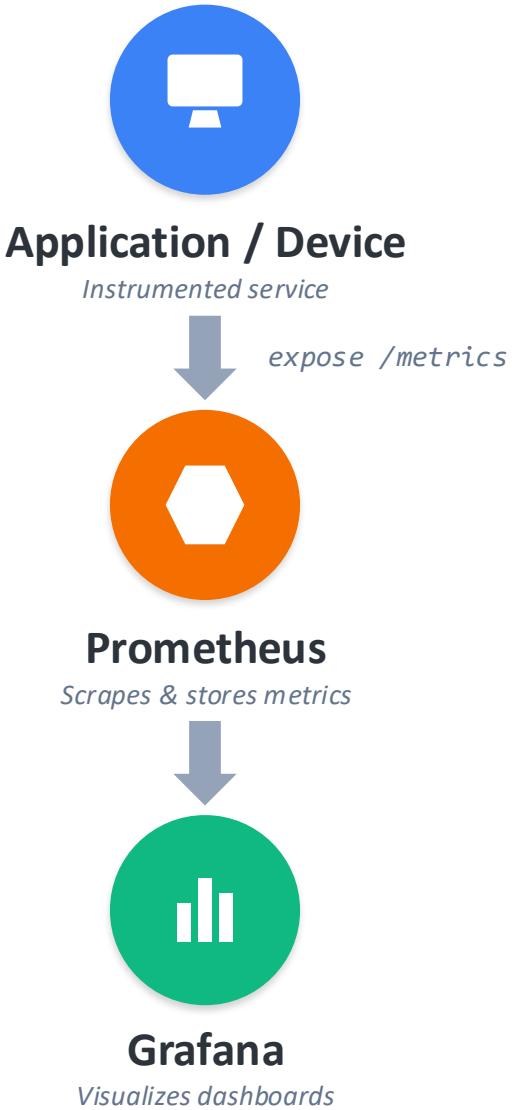
Prometheus Data Source

Time-series metrics · PromQL

- Prometheus scrapes and stores time-series metrics from configured targets
- Query language: PromQL — functional, built for time-series aggregation
- In Docker Compose, the URL is: http://prometheus:9090
- Explore → Code mode → type PromQL to see live metric data

Prometheus Data Source

Time-series metrics · PromQL



How Prometheus Stores Data

Time-Series Database (TSDB)

(timestamp, metric, labels, value)

How Prometheus Stores Data

Time-Series Database (TSDB)

(timestamp, metric, labels, value)

2026-05-06 10:00 electricity_usage_watts{room="meeting"} 452

How Prometheus define Schema

Time-Series Database (TSDB)

metric_name{label="value"} number

temperature_celsius{room="server-room"} 23.5

How Prometheus define Schema

Time-Series Database (TSDB)

```
http_requests_total{
  method="GET",
  status="500",
  service="payment"
}
```

How Prometheus Query Data

Time-series metrics · PromQL

```
cpu_usage_percent

http_requests_total{status="500"}

avg(cpu_usage_percent)

rate(http_requests_total[5m])
```

Prometheus Data Source

STEPS

1. Start the lab stack with docker compose up -d
2. Grafana → Connections → Add new data source → Prometheus
3. Set URL to http://prometheus:9090 → Save & test
4. Open Explore and run: up and prometheus_build_info

EXPECTED OUTCOMES

- ✓ Prometheus data source added
- ✓ Connection test passed (green)
- ✓ First PromQL query result visible in Explore

InfluxDB Data Source with Flux

InfluxDB 2.x · Flux query language

- InfluxDB 2.x uses Flux — a functional scripting language for data pipelines
- Required connection fields: URL, Organization, Default Bucket, and Token
- Select 'Flux' as the query language when configuring the data source
- Compares a different paradigm alongside PromQL in the same Grafana UI

InfluxDB + Flux Data Source

STEPS

1. Run the seed script: `./scripts/seed-influx.sh`
2. Grafana → Connections → Add InfluxDB data source (Flux mode)
3. Fill in URL, Org, Token, and Default Bucket
4. Query the `lab_metrics` measurement in Explore

EXPECTED OUTCOMES

- ✓ InfluxDB data seeded successfully
- ✓ InfluxDB data source connected (Flux mode)
- ✓ Basic Flux query returns results in Explore

Section 5

PromQL Basics

Selectors · Functions · Aggregations

What is PromQL?

The query language built into Prometheus

- Functional language for selecting and aggregating time-series data
- Every expression returns an instant vector, range vector, or scalar
- Used in Grafana Explore, panel editors, and alert rule definitions
- Not SQL — designed around labelled metric streams and time windows

PromQL: Basic Metric Selectors

Select, filter, narrow

- up — 1 = target reachable, 0 = down
- {job="prometheus"} — label selector to filter results
- up{job="prometheus",instance="…"} — combine multiple selectors
- prometheus_build_info — build metadata per Prometheus instance
- [5m] range selector — look back 5 minutes of samples

PromQL: Useful Functions & Aggregations

Four patterns that cover most real-world queries

- rate(metric[5m]) — per-second rate; use for counter metrics
- increase(metric[1m]) — total counter change; useful in alert thresholds
- sum by (label) (metric) — aggregate across instances grouped by label
- avg_over_time(metric[5m]) — average gauge value over a rolling window
- Rule: never plot raw counter values — always use rate() or increase()

PromQL in Grafana Explore

Builder mode vs Code mode

- Select Prometheus as the data source in the Explore page
- Builder mode — visual form for discovery and learning
- Code mode — type raw PromQL for complex expressions
- Time range picker to zoom in; Query Inspector to view raw JSON

Section 6

Queries and Panels

Explore → Panel → Dashboard

Queries in Grafana

Start with the question, not the chart type

- Begin in Explore — experiment freely before committing to a panel
- Choose the data source first, then write a query that answers a question
- A good query in Explore transfers directly to a dashboard panel
- The query is the foundation — visualization follows from the data

Designing and Building Panels

Match the visualization to the question

- Time Series — trends and changes over a time range
- Stat — a single current value read at a glance
- Table — comparison across multiple dimensions or instances
- Panel title, legend labels, and layout matter as much as the query

Queries and Panels

STEPS

1. Import the starter dashboard (dashboards/starter.json)
2. Add a Time Series panel: up{job='prometheus'}
3. Add a Stat panel: prometheus_build_info
4. Set panel titles, configure legends, and adjust layout

EXPECTED OUTCOMES

- ✓ First real dashboard built
- ✓ Time Series vs Stat panel differences understood
- ✓ PromQL queries rendered on dashboard panels

Exercises/04-queries-and-panels.md

Section 7

Advanced Dashboard Features

Variables · Templating · Drill-Down Links

Dashboard Variables

Eliminate hard-coded values · Enable dynamic filtering

Variable Dropdowns

$job = prometheus

$instance = localhost:9090

$interval = 5m

↓

```
rate(http_requests_total{job="$job", instance="$instance"}[$interval])
```

↓

All panels update simultaneously

Panel A

Panel B

Panel C

Panel D

Dashboard Variables with InfluxDB

STEPS

1. Seed multi-region InfluxDB data with ./scripts/seed-influx.sh
2. Create a 'service' Query variable using a Flux query
3. Create a 'region' variable chained to the selected service
4. Update panel queries to use $service and $region

EXPECTED OUTCOMES

- ✓ Query variables populated from Flux live data
- ✓ Chained variable filters correctly on selection
- ✓ Dashboard reacts dynamically to dropdown changes

Templating: Reusable Dashboards

Query · Custom · Interval variable types

- Query variable — values come from real metric labels (fully dynamic)
- Custom variable — a fixed list defined by the dashboard author
- Interval variable — control the time step/resolution of queries
- Panel Repeat — auto-expand one panel definition into N panels per variable

Templating

STEPS

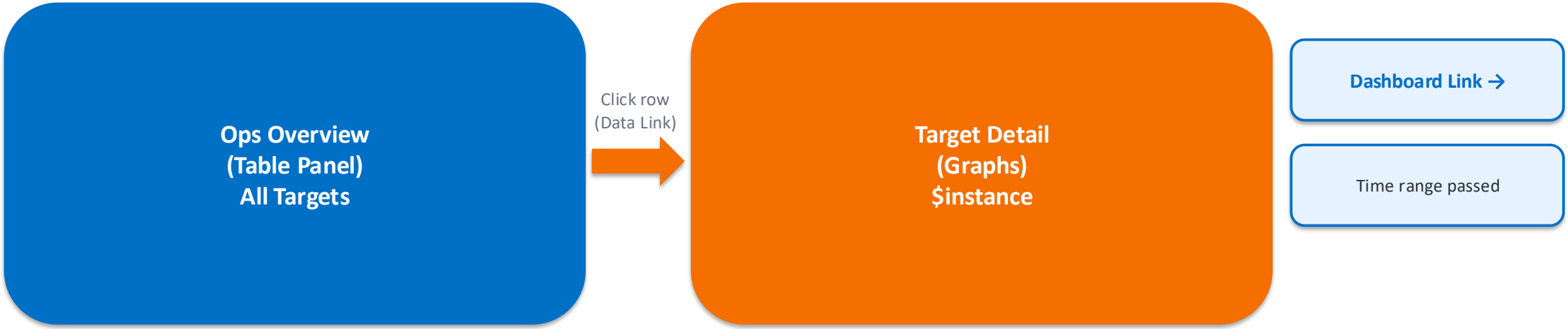
1. Import the starter.json dashboard skeleton
2. Create a 'service' Query variable (from Prometheus label values)
3. Create a custom 'interval' variable: 1m, 5m, 15m, 1h

EXPECTED OUTCOMES

- ✓ Query variable drives panel content dynamically
- ✓ Custom interval variable controls query resolution
- ✓ Panel Repeat expands one panel per service value

Interactive Dashboards

Dashboard Links · Data Links · Drill-Down Flow



Context preserved across navigation:

- Time Range
- Variable Values
- Selected Data Point

Interactive Dashboards

STEPS

1. Create 'Ops Overview' with Table panel: sum by (job, instance) (up)
2. Create 'Target Detail' dashboard with time series graphs
3. Add a Dashboard Link from Overview → Detail (preserve time range)
4. Add a Data Link on the table row to pass $instance as variable

EXPECTED OUTCOMES

- ✓ Dashboard link navigates and preserves time range
- ✓ Data link in Table opens correct detail view
- ✓ Variable ($instance) context carried between dashboards

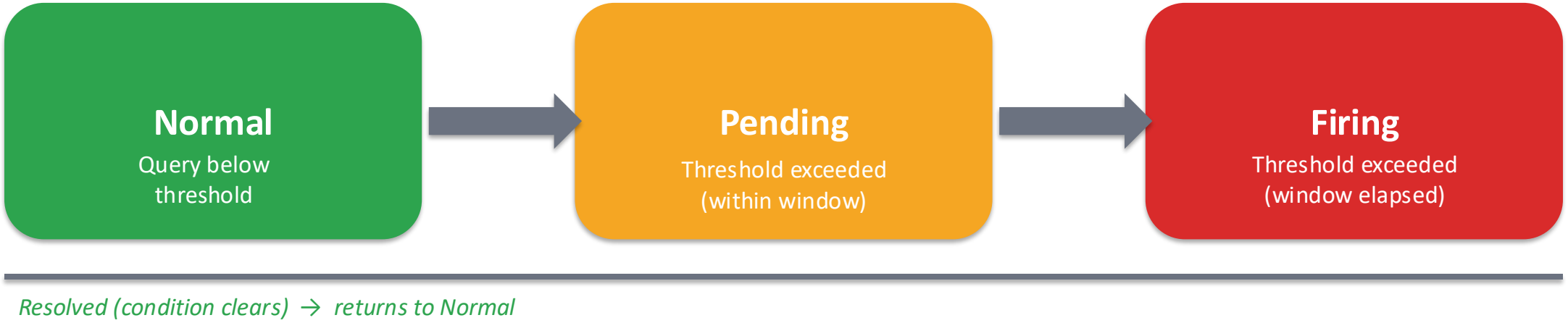
Section 8

Working with Alerting

Rules · Thresholds · State Transitions

Grafana Alerting System

Evaluate · State Machine · Notify



Evaluation Interval
How often rule is checked

Pending Period
Prevents alert flapping

Silences
Suppress notifications

Setting Up Alert Rules

Query → Condition → Pending → Fire

- Write a PromQL query that measures something operationally meaningful
- Set a threshold condition — e.g. value > 10
- Evaluation interval — how often the rule is checked (e.g. every 1m)
- Pending period — how long condition must hold to prevent flapping

Alerting Basics

STEPS

1. Create an alert rule: `increase(demo_requests_total[1m]) > 10`
2. Set evaluation interval to 1m and pending period to 2m
3. Run `./generate-traffic.sh` to trigger the condition
4. Watch the state transition in the Alert Rules UI

EXPECTED OUTCOMES

- ✓ Alert rule created with threshold condition
- ✓ Traffic script successfully triggers the alert
- ✓ Normal → Pending → Firing transition observed in UI

Section 9

Grafana Plugins

Data Source · Panel · App · TypeScript + React

What are Grafana Plugins?

Extend the platform for any specialized need

Connect Grafana to
any data backend.
Add new query editors.

Data Source Plugin

Create new
visualization types.
Custom React components.

Panel Plugin

Bundle data sources,
panels & pages into
a complete product.

App Plugin

Scaffold any type with: npx @grafana/create-plugin@latest

Installing and Configuring Plugins

Public catalog · Local development · Version compatibility

- Browse the public catalog at grafana.com/plugins
- Install via: grafana-cli plugins install <plugin-id>
- For development: mount the dist/ folder via Docker Compose volume
- Verify permissions, settings, and Grafana version compatibility after install

Developing a Basic Panel Plugin

Scaffold · Develop · Live Reload

- Scaffold with: npx @grafana/create-plugin@latest
- Develop in TypeScript + React — same stack as Grafana core
- Run npm run dev to start the file watcher (live reload)
- Mount dist/ in docker-compose.yml — no container rebuild on changes

Panel Plugin Basics

STEPS

1. Run `npx @grafana/create-plugin@latest` in labfiles/09-panel-plugin-basics/workspace/
2. Select 'Panel' plugin type and enter a plugin name
3. Run `npm install && npm run dev`
4. Edit SimplePanel.tsx, save, and confirm live reload in Grafana

EXPECTED OUTCOMES

- ✓ Plugin scaffolded and installed successfully
- ✓ Grafana detects and loads the new plugin
- ✓ Code edit reflected in rendered panel (live reload working)

Section 10

Summary & Next Steps

Recap · Q&A · Keep building

41

Course Summary

What you built over two days

- Launched Grafana in Codespaces using Docker Compose
- Connected Prometheus (PromQL) and InfluxDB (Flux) as data sources
- Built dashboards with panels, variables, and templated views
- Created a drill-down flow with Dashboard and Data Links
- Configured alert rules and observed state transitions
- Scaffolded and rendered a custom panel plugin

Q&A and Next Steps

Keep building beyond the course

- Swap in your team's real Prometheus / InfluxDB data sources
- Build dashboards that answer your team's actual operational questions
- Extend the panel plugin with custom options and data transformations
- Explore Grafana Loki (logs) and Tempo (traces) for full-stack observability

Thank you

ธีรเศรษฐ์ จิรภัทร์ชาญเดช (อ.พล)

Microsoft® MVP Most Valuable Professional

- Founder, Nextflow.in.th
- Community Leader, Global AI Bangkok
- AI Instructor & Consultant

  – SCBx, Boon Rawd Brewery, สำนักงานนวัตกรรมแห่งชาติ, U.S. Embassy Bangkok

Contact me 😎

[Slide Title — click to customise]

[Content area — add text, diagrams, images, or charts here]